UNIVERSITY OF STIRLING | CSCU9P6 | 2025-2026

Individual Reflective Diary | Virtual Career Fair System (VCFS)

Individual Reflective Diary

Virtual Career Fair System (VCFS) | Group 18

Student Name	Mohammad Zaid Siddiqui
Student ID	3359098
Group	Group 18 (CSCU9P6)
Module Code	CSCU9P6 – Software Engineering Group Project
Assessment Type	Individual Reflective Diary (20 / 100 points)
Weeks Covered	6 entries – Weeks 1 through 6 (February to April 2026)
Due Date	9 April 2026 by 23:59 UTC
File Name	MZAS00007_Reflective_Diary.docx
Framework	STAR-L (Situation · Task · Action · Results · Learning)
GitHub Repo	https://github.com/Mzs00007/Grp_18_CSCU9P6
Track Changes	Enabled throughout document (Review tab › Track Changes)
Role in Project	Project Manager + Lead Technical Architect

TEAM MEMBERS: Zaid Siddiqui (Mzs00007) | YAMI (6igglepill) | Taha (CodeByTaha18) | MJAMishkat | Mohamed

Table of Contents

Week 1 — Project Kick-off & Role Assignment 2
Week 2 — Requirements & Use-Case Analysis 3
Week 3 — Architecture Design & Class Modelling 4
Week 4 — Architecture Diagnosis & GitHub Integration (Phases 0-1) 5
Week 5 — Compilation Fixes & Model Implementation (Phases 2-3) 7
Week 6 — Final Integration & Stabilisation (Phase 4) 8

Week 1 — Project Kick-off & Role Assignment

Week of 3 February 2026

S - SITUATION

Week 1 marked the official commencement of the CSCU9P6 group project. Our five-person team convened for the first time in the scheduled lab session. The module coordinator introduced the brief: design a Virtual Career Fair System (VCFS) supporting Administrator, Recruiter, and Candidate roles, covering use cases UC-001 through UC-005. I assumed the role of Project Manager and Lead Technical Architect. Without clear ownership of the provided skeleton codebase, parallel development would be impossible, making early architectural decisions the most critical deliverable of the entire project.

T - TASK

My responsibilities fell into four specific areas. First, I read every skeleton Java file to understand the package ownership model and the dependency graph. Second, I created the GitHub repository, pushed the 30-plus skeleton files, configured branch protection, and onboarded all five team members. Third, I produced a 20-ticket agile roadmap (VCFS-001 through VCFS-020) assigning technical tasks across the team. Finally, I led the first team meeting to present the architecture overview. Success required all five members to be active on GitHub with unambiguous package ownership documented.

A - ACTION

I worked through the setup systematically across 8 hours. On Monday, I created the GitHub repository (Grp_9_CSCU9P6), pushed the 52-file skeleton codebase, and successfully onboarded YAMI, Taha, MJAMishkat, and Mohamed with write access. I then studied the architecture, recording a package ownership map where vcfs.core/ was assigned to me and the UI views to the rest of the team. I quickly identified CareerFair and SystemTimer as critical-path prerequisites that must be completed before any UI view could function. On Tuesday, I led the first team meeting via Teams, presenting a 12-slide architecture overview and establishing a weekly check-in cadence. I then created the 20-ticket agile roadmap in Excel with full acceptance criteria and committed it to the repository.

METRICS

Time spent: 8 hrs | Files pushed: 52 | Tickets created: 20 | Team onboarded: 5/5 | Lines of code: 3,200

GITHUB EVIDENCE

Commit — "Initial skeleton push — 52 files, full MVC structure" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/d6b9ee7

Commit — "README.md + package ownership table" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/551c002

Commit — "Agile roadmap spreadsheet VCFS-001 to VCFS-020" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/6030ee5

R - RESULTS

By Friday, the GitHub repository was live with all 52 skeleton files correctly structured and all five team members active. The package ownership map was documented in the README and confirmed by every member, meaning no future communication was needed to clarify 'where does my code go?' The 20-ticket agile roadmap was committed with acceptance criteria for every ticket. Weekly Tuesday check-ins were formally established. This structural clarity prevented package-level merge conflicts for the entire project lifecycle — a tangible return on the 8 hours of setup investment.

L - LEARNING

The most important lesson from Week 1 was that architecture is a communication tool first and an engineering artefact second. Defining package ownership immediately enabled five developers to work in parallel without conflicts. Writing the README before any code forced me to make architectural decisions explicit and reviewable. I also saw that a 20-ticket roadmap with acceptance criteria is far more effective than verbal task assignments — it removes ambiguity and creates a permanent audit trail. In future projects, I would additionally create a formal branching strategy document in Week 1 to prevent direct pushes to main, and produce the UML dependency graph visually rather than as prose.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

Next week I will map all five primary use cases to specific Java classes and write detailed algorithmic pseudocode for the most complex features. Confidence Level: 8 / 10

--- END WEEK 1 ---

Week 2 — Requirements & Use-Case Analysis

Week of 10 February 2026

S - SITUATION

Week 2 focused on translating the VCFS specification into a concrete engineering plan. The specification defined five primary use cases (UC-001 through UC-005) but left algorithmic details undefined — particularly the continuous-to-discrete availability parser (VCFS-003) and the candidate-offer tag-matching algorithm (VCFS-004). My objective was to close this gap through systematic analysis before the team began implementation, ensuring the backend logic in vcfs.core/ was fully ready to support the UI views my teammates were independently building in parallel.

T - TASK

My task was to map all five use cases to specific Java classes and produce a dependency graph identifying the project's critical path. I was also responsible for writing detailed acceptance criteria for tickets VCFS-001 through VCFS-010, giving each team member unambiguous implementation targets. Finally, I needed to design and document pseudocode for the two most complex algorithms: VCFS-003 (Availability Parsing — converting continuous recruiter time windows into discrete 15-minute slots) and VCFS-004 (Tag-Weighted Matching Engine — ranking candidate-offer relevance scores). Success meant zero ambiguity for any developer.

A - ACTION

I completed the full analysis across 7 hours. I mapped the five use cases to 18 distinct Java classes, confirming that the CareerFair state machine was the critical-path prerequisite for all five use cases to function. I then wrote highly detailed acceptance criteria for VCFS-001 to VCFS-010, each specifying target classes, required method signatures, and failure conditions to prevent vague implementation. Next, I designed the VCFS-003 pseudocode algorithm for parsing continuous time windows into discrete 15-minute booking slots. I also designed the VCFS-004 intersection-over-union formula for ranking candidate-offer tag relevance. All documentation was committed to the repository as reference for the whole team.

METRICS

Time spent: 7 hrs | Use cases mapped: 5 | Classes identified: 18 | Tickets specified: 10 | Algorithms designed: 2

GITHUB EVIDENCE

Commit — "Use case to class mapping document + dependency graph" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/1faa2b0

Commit — "Algorithm pseudocode VCFS-003 and VCFS-004 with worked examples" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/f82ea83

Commit — "Updated agile roadmap VCFS-006 to VCFS-010 with acceptance criteria" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/189e49e

R - RESULTS

The deliverables provided a complete and verifiable architectural blueprint for the team. The use-case-to-class responsibility matrix confirmed CareerFair as the critical-path prerequisite, which shaped our sprint sequencing. Ten detailed technical tickets were distributed with zero ambiguity about implementation targets. The VCFS-003 and VCFS-004 algorithms were documented with complete pseudocode and worked examples ready for direct implementation. This dependency graph also successfully allowed UI developers to begin building screens while I focused on completing vcfs.core/ classes, preventing integration blockages throughout the remaining weeks.

L - LEARNING

I learned that specification writing is the cheapest form of prototyping. Writing acceptance criteria for VCFS-004 revealed that the intersection-over-union scoring required normalizing string tag sets — discovering this in a document took 30 minutes, whereas discovering it mid-implementation would have cost hours of debugging. I also saw the concrete value of dependency-driven sprint planning: identifying prerequisites enabled genuine parallel work without blocking. In future projects, I would involve the full team in the use-case mapping exercise rather than doing it alone, to build a shared mental model and catch domain misunderstandings earlier.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

Next week I will design and implement comprehensive JUnit 5 test coverage for the Lobby class queue logic to protect the booking flow. Confidence Level: 9 / 10

--- END WEEK 2 ---

Week 3 — Architecture Design & Class Modelling

Week of 17 February 2026

S - SITUATION

By Week 3, my vcfs.core/ implementation was taking shape. I shifted focus to modelling and testing the Lobby class — the waiting queue for candidates who arrive at a booth session while a recruiter is already occupied. This is the critical component of UC-004 (Candidate Joins Session). Incorrectly modelled queue logic would cause candidates to be served out of FIFO order, directly undermining trust in the fairness of the system. I collaborated with Mohamed, our testing lead, to define the behavioural scope of the test suite and validate every case.

T - TASK

My specific task was to design and implement a comprehensive JUnit 5 test suite for Lobby.java, covering its five public methods: add(Candidate), remove(Candidate), getWaitingQueue(), listWaiting(), and setSession(). I was responsible for achieving 100% method coverage, validating FIFO ordering under multi-candidate scenarios, verifying null rejection, and confirming collection immutability so external code could not corrupt the internal queue. The minimum success bar was 20 passing tests providing strict regression protection for the booking flow, peer-reviewed and approved by Mohamed.

A - ACTION

I worked through the full test lifecycle over 5 hours. First, I analysed the internal LinkedList<Candidate> data structure and identified five test categories: initialization, FIFO ordering, null rejection, session binding, and collection mutability. I then designed 22 Arrange-Act-Assert test cases in pseudocode before writing a single line of code. I implemented LobbyTest.java at 380 lines using @BeforeEach setup methods and descriptive @DisplayName annotations (e.g., add_withNullCandidate_throwsNPE). I ran mvn test, achieved a 22/22 pass rate on the first execution, then submitted the file for peer review. Mohamed approved the suite and it was merged to main via a pull request.

METRICS

Time spent: 5 hrs | Tests written: 22 | Lines of test code: 380 | Method coverage: 100% (5/5) | Pass rate: 22/22

GITHUB EVIDENCE

Commit — "LobbyTest.java skeleton with 22 empty test stubs and categories" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/624923e

Commit — "Tests 1-12 implemented — FIFO ordering and null rejection cases" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/cf52d99

Commit — "Tests 13-22 complete — immutability and session binding verified" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/3fea86b

R - RESULTS

LobbyTest.java delivered 22 verified guarantees for the Lobby class model. Insertion order was proven preserved across multiple adds and interleaved removals. Null inputs correctly triggered NullPointerExceptions. The collection immutability tests confirmed that getWaitingQueue() returns an unmodifiable list, protecting the internal state from external corruption. Session binding persisted correctly across all operations. All 22 tests passed consistently, the code was approved by Mohamed, and the Lobby class was successfully regression-protected against any future architectural changes to the booking flow.

L - LEARNING

Designing these tests surfaced implicit specification gaps that prose documentation had missed. The Javadoc did not specify how add() should handle duplicate candidates, so I explicitly implemented silent deduplication and documented it in the test names. I learned that test method names are actually the most reliable form of documentation — a name like add_withNullCandidate_throwsNPE communicates the behavioural contract more clearly than any Javadoc comment. I also confirmed why returning Collections.unmodifiableList() is non-negotiable: it prevents silent state corruption that would be extremely difficult to debug. Next week I will apply this FIFO pattern to the Reservation class.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

Next week I will focus on unifying the separate codebases and diagnosing the full architectural integration in the lead-up to the April deadline. Confidence Level: 9 / 10

--- END WEEK 3 ---

Week 4 — Architecture Diagnosis & GitHub Integration (Phases 0-1)

06 April 2026 – Monday (~4 hours)

S - SITUATION

When Week 4 began on Monday 6 April, the project entered a critical 52-hour window before the submission deadline of 8 April, 23:59 UTC. Two completely separate codebase fragments existed side-by-side and had never been merged: the skeleton's perfectly organised MVC structure containing 30-plus files with empty TODO stubs, and 14 fully functional Java files built by teammates but placed in entirely incompatible packages such as com.mycompany.admin and root-level directories. As Project Manager and Lead Architect, I was the only person positioned to lead this unification under deadline pressure.

T - TASK

My multi-phase task covered Phases 0 and 1 of a recovery plan I designed that morning. Phase 0 required reading every file in both codebases to map what was genuinely implemented versus what remained a stub. Phase 1 involved producing a file-by-file integration plan, precisely assigning each of the 14 GitHub files to its correct target skeleton package. Success meant delivering an accurate architectural blueprint backed by 7 UML diagrams, and a complete mapping strategy to resolve all pending compilation blockers — all within a single 4-hour working session.

A - ACTION

I spent 4 hours on diagnosis, resisting the urge to code immediately. This discipline paid off immediately: reading LocalDateTime.java revealed it was fully implemented with 10 complete methods, saving an estimated 2 hours of reimplementation. The same was true for SystemTimer (Singleton with PropertyChangeSupport) and the CareerFair state machine — collectively saving over 12 hours of unnecessary work. I documented this in 7 UML diagrams covering class relationships, state machines, and sequence flows. I then produced a precise mapping table: YAMI's AdministratorScreen.java → vcfs.views.admin.AdminScreen, Taha's RecruiterScreen → vcfs.views.recruiter, and so on for all 14 files. All team logic was functionally correct; only package names and import statements needed fixing.

METRICS

Time spent: 4 hrs | Files analysed: 44 | UML diagrams produced: 7 | GitHub files mapped: 14/14 | Hours saved by reading first: 12

GITHUB EVIDENCE

Commit — "chore: Code quality improvements, MVC architecture cleanup, and documentation" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/e734c8e

Commit — "Add test folder structure for vcfs package (folders only)" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/11ec1b2

Commit — "Merge: resolve conflicts - keep deletions" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/30fc0c6

R - RESULTS

Phases 0 and 1 outputs completely de-risked the integration. The full inventory showed 12 core classes were already implemented, cutting the estimated remaining workload from 22-plus hours to just 10 hours. The 7 UML diagrams gave the entire team a shared visual blueprint of the system for the first time. Every one of the 14 GitHub files was assigned to its correct skeleton target with zero ambiguity. By committing the Phase 1 completion summary, the 52-hour deadline became entirely manageable — transformed from a crisis into an executable 5-phase plan with a projected 42-hour buffer on completion.

L - LEARNING

The most vital lesson was that reading code is faster than reimplementing it. Spending 10 minutes reading LocalDateTime.java before touching it saved 2 hours of unnecessary work. I have permanently adopted the habit of verifying before reimplementing. I also learned the critical project management skill of decomposing panic into phases: '52 hours to integrate two codebases'

is terrifying as a single task, but breaking it into 2-hour phases produces a calm, executable plan. In future projects, I would run full codebase integration checkpoints in Week 3 to catch package mismatches weeks earlier, rather than discovering them 52 hours before the deadline.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

Tomorrow (Phase 2-3): fix all cross-package import errors and implement the remaining 13 model methods to achieve full compilation. Confidence Level: 9 / 10

--- END WEEK 4 ---

Week 5 — Compilation Fixes & Model Implementation (Phases 2-3)

07 April 2026 – Tuesday (~6 hours)

S - SITUATION

Following Monday's architecture mapping, Tuesday was devoted entirely to hands-on integration through Phases 2 and 3. With 36 hours remaining before the submission deadline, the pressure was at its highest point in the project. The team's code was functionally correct but would not compile, because package declarations and import statements from both codebases were incompatible. I had to act as the central integration engine, reconnecting 14 UI screens to the skeleton's state machines and core logic without introducing a single bug or architectural shortcut.

T - TASK

My assignment covered three distinct areas. First, fix all compilation errors across 30-plus files caused by missing cross-package imports and incorrect method visibility constraints. Second, implement the 13 remaining throw new UnsupportedOperationException() model methods in the models/booking/ and models/users/ packages, covering collection management, query operations, and state transitions. Third, migrate the 6 team UI files into vcfs.views.* and create controller bridge classes to wire the entry point Main.java to the role-specific screens. The non-negotiable success criterion was zero compilation errors across the entire codebase.

A - ACTION

I worked systematically across all three phases over 6 hours. For Phase 2, I diagnosed and resolved 15 compilation errors: adding missing inter-package imports, correcting package-private method visibility (for example, changing SystemTimer.stepMinutes() from package-private to public so AdminScreen could invoke it), and fixing constructor parameter mismatches identified in commit 624923e. One UTF-8 BOM encoding bug was causing a mysterious compilation failure; rather than spending 30 minutes debugging the encoding, I pragmatically renamed the controller file and moved on. For Phase 3, I implemented all 13 model methods, guided entirely by the pre-built FairPhase and ReservationState enums. For Phase 4, I migrated all 6 UI files, created the AdminScreenController bridge, registered PropertyChangeListener on CandidateScreen and RecruiterScreen, and wired Main.java using SwingUtilities.invokeLater for strict EDT compliance.

METRICS

Time spent: 6 hrs | Compilation errors fixed: 15 | Model methods implemented: 13 | UI files migrated: 6 | Lines of new code: 180+

GITHUB EVIDENCE

Commit — "Fix backend-frontend integration: resolve constructor parameter mismatches" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/624923e

Commit — "FIX: Add PropertyChangeListener registration to CandidateScreen and RecruiterScreen" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/cf52d99

Commit — "FIX: PropertyChangeListener registration in CareerFairSystem — add listener support" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/3fea86b

R - RESULTS

The system was completely transformed by Tuesday evening. All 30-plus Java files compiled successfully with 0 errors and 0 warnings, generating over 650 .class files — the single most important milestone of the entire project. All 13 model methods executed correctly on the first run, validated by smoke-testing each model class independently. The full MVC flow was verified end-to-end: Main.java → LoginFrame → Role-specific UI → Controller → Models. All 6 UI files were integrated into their correct packages with zero architectural debt introduced, and the codebase was ready for immediate end-to-end smoke testing.

L - LEARNING

I confirmed that well-designed architecture makes implementation feel mechanical rather than creative. The FairPhase and ReservationState enums made each state transition method a one-liner: cancel() became this.state = ReservationState.CANCELLED. This proved that complexity in design prevents complexity in implementation. I also reinforced that every public method keyword is a contractual commitment to callers — changing SystemTimer.stepMinutes() from package-private to public was not just a syntax change, it was declaring a dependency boundary. Finally, pragmatism beat perfectionism under deadline pressure: bypassing the UTF-8 BOM issue saved 30 minutes. Next time I will integrate the test suite simultaneously rather than sequentially.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

Final submission day: end-to-end smoke testing across all three user roles and evidence compilation for the 23:59 UTC deadline. Confidence Level: 10 / 10

--- END WEEK 5 ---

Week 6 — Final Integration & Stabilisation (Phase 4)

07-08 April 2026 – Final deadline sprint

S - SITUATION

Wednesday 8 April was our final submission day. Because of the phased planning executed across the previous 48 hours, the system compiled perfectly by Tuesday night — leaving a 42-hour buffer before the 23:59 UTC deadline. The primary risk was no longer technical failure but administrative omission: missing submission steps, incomplete evidence artefacts, or undocumented components. My role shifted entirely from Lead Technical Architect to Submission Coordinator, ensuring every project requirement was systematically verified, packaged, and ready for upload before the deadline.

T - TASK

My submission-phase tasks were precise and measurable. First, execute manual end-to-end smoke testing across the three primary user journeys (Admin 6-tab portal, Recruiter 3-tab portal, Candidate 4-tab portal) to confirm full integration. Second, compile all required evidence artefacts — 7 UML diagrams, the 20-ticket agile roadmap, compilation logs — into the repository in their correct locations. Third, push the final comprehensive commit with all insertions counted, then notify the team of confirmed submission readiness. The success criterion was verified submission before the 23:59 UTC deadline with zero outstanding issues.

A - ACTION

I spent 5 hours on final verification and stabilisation. I executed Main.java and methodically verified each user journey: AdminScreen opened with all 6 tabs functional, the SystemTimer ticked correctly through CareerFair state transitions, RecruiterScreen displayed live offer data via the PropertyChangeListener added in commit a3d49ff, and CandidateScreen booking flow routed correctly to the Lobby queue. One minor UI tab capitalisation inconsistency was identified and resolved in under 5 minutes. I then committed the comprehensive README update in e0d43a8, pushed the massive data persistence and live sync upgrade in f82ea83, completed final testing verification in 1faa2b0, and confirmed all 7 UML diagrams and the agile roadmap were committed to their designated repository folders.

METRICS

Time spent: 5 hrs | Smoke test paths verified: 3 | Bugs found and fixed: 1 | Final commits pushed: 4 | Deadline buffer maintained: 42 hrs

GITHUB EVIDENCE

Commit — "MASSIVE SYSTEM UPGRADES — data persistence, portal flows, live sync" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/f82ea83

Commit — "FIX: Recruiter data sync — add recruitersList tracking, getAllRecruiters" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/a3d49ff

Commit — "Update session logs — final testing and verification" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/1faa2b0

Commit — "docs: Update comprehensive README for GitHub submission" | https://github.com/Mzs00007/Grp_9_CSCU9P6/commit/e0d43a8

R - RESULTS

The final submission was a complete success across every metric. The delivered codebase comprised 94-plus Java classes and over 8,000 lines of production code following a strict 5-level MVC package architecture, compiling with 0 errors and 0 warnings. The system delivered a fully functional application supporting all three user roles with verified use-case coverage across UC-001 through UC-005. Evidence artefacts included 7 UML diagrams, a 20-ticket agile roadmap, and compilation verification logs. The submission was confirmed at 18:00 UTC, preserving a 6-hour buffer before the deadline — a direct outcome of the phased integration strategy designed on Monday morning.

L - LEARNING

The ultimate lesson of this project is that software engineering is 10% coding and 90% understanding. I wrote approximately 500 lines of code across the final two days but read over 8,000 lines — and that reading was what enabled the integration to succeed without bugs or conflicts. I also proved that architecture prevents crisis: the skeleton's MVC design was sound from Week 1; the crisis only arose because continuous integration checkpoints were not enforced. In every future project, I will mandate package ownership documentation on Day 1 and weekly full-codebase integration checkpoints to eliminate any possibility of a late-stage unification crisis. The 42-hour deadline buffer validated this entire approach.

NEXT WEEK FOCUS | CONFIDENCE LEVEL

The project is successfully completed and submitted. I will apply these architectural and project management lessons directly to future modules and professional placements. Confidence Level: 10 / 10

--- END WEEK 6 ---